Security — E-Commerce Platform

1. CI Scanning (GitHub Actions)

All scanning runs in CI before images reach ECR. ArgoCD only deploys verified images.

Dependency Scanning

- **Python**: `pip-audit` or `safety` — checks `requirements.txt` against known CVE databases
- **Node.js**: `npm audit` — checks `package-lock.json` for vulnerable packages

Docker Image Scanning

- **Trivy** (Aqua Security) — scans the built image for OS-level CVEs in system packages and libraries. Fails pipeline on CRITICAL/HIGH severity.

Code Security

- **Bandit** (Python) — finds hardcoded passwords, SQL injection, unsafe `eval()` calls
- **Semgrep** — generic rule-based scanner for all languages. Catches logic bugs dependency scanners miss

Secret Leak Detection

- **Gitleaks** — scans every commit for accidentally committed secrets (API keys, tokens, passwords). Runs on every push.

CI Pipeline Flow

```
Push → GH Actions
  ├── gitleaks (secrets check)
  ├── npm audit / pip-audit (dependency CVEs)
  ├── build Docker image
  ├── trivy scan (image CVEs)
  ├── push to ECR (only if all pass)
  └── update helmfile → commit → push
                          ↓
                  ArgoCD syncs to cluster
```

2. Secrets Management

Problem

Secrets (`SECRET_KEY`, DB passwords) cannot live in Git — even Base64 is not encryption. Anyone with repo access can decode them.

Solution: External Secrets Operator (ESO)

- Store secrets in **AWS Secrets Manager**
- ESO runs in-cluster and syncs secrets from AWS into Kubernetes `Secret` objects
- Git only contains **references** to which secret to fetch, never the actual value

```
# In Git (safe):
apiVersion: external-secrets.io/v1beta1
kind: ExternalSecret
metadata:
  name: django-secret
spec:
  secretStoreRef:
    name: aws-secretsmanager
    kind: ClusterSecretStore
  target:
    name: user-management-secret
  data:
    - secretKey: SECRET_KEY
      remoteRef:
        key: ecommerce/user-management/secret-key
```

Alternative: Sealed Secrets

- Encrypt secrets in Git with a public key
- Only the Sealed Secrets controller in-cluster can decrypt with the private key
- Encrypted values are safe to commit to Git

3. ArgoCD Security

Git Access

- ArgoCD needs **read-only** access to the Git repo

- Use a **deploy key** (SSH) or **fine-grained personal access token** with `contents:read` only
- No write access needed — CI handles commits to `helmfiles/`

Cluster Access

- ArgoCD runs as a ServiceAccount with RBAC roles scoped to the `ecommerce` namespace
- Cannot access `kube-system` or other namespaces
- Cannot modify cluster-wide resources (CRDs, StorageClasses, etc.)

Sync Policies

- `prune: true` — deletes resources removed from Git (prevents orphaned resources)
- `selfHeal: true` — reverts manual `kubectl edit` changes to match Git (prevents drift)

4. ECR & Image Security

Image Pull

- EKS worker nodes have IAM role with `ecr:GetAuthorizationToken` and `ecr:BatchGetImage`
- No access keys stored on nodes — uses instance metadata

Lifecycle Policy

- Keep only 1 tagged image per repo
- Untagged images expire after 1 day
- Reduces storage cost and attack surface

Image Tagging

- Images tagged with Git SHA (`v1.0.0-abca63e`) — immutable, traceable
- No `latest` tag in production — always use specific version

5. Network Security

Current State

- All services: `ClusterIP` — internal only, no external access

- Databases: `ClusterIP` — only accessible from within `ecommerce` namespace
- No Ingress configured yet

When Ingress is Added

- AWS Load Balancer Controller creates ALB with HTTPS (ACM certificate)
- HTTP redirects to HTTPS automatically
- Security groups on ALB restrict source IPs if needed
- NetworkPolicies can restrict pod-to-pod traffic (e.g., only microservices can reach databases)

Example NetworkPolicy

```yaml
apiVersion: networking.k8s.io/v1
kind: NetworkPolicy
metadata:
  name: db-restrict
  namespace: ecommerce
spec:
  podSelector:
    matchLabels:
      app: mongodb
  ingress:
    - from:
        - podSelector:
            matchLabels:
              app: product-catalog
  # Only product-catalog pods can reach MongoDB
```

6. IAM & RBAC

EKS Access

- Root IAM user has `AmazonEKSClusterAdminPolicy` via access entry
- CI uses OIDC (no long-lived AWS credentials)
- Node IAM role grants only ECR pull + CloudWatch logs

Kubernetes RBAC

- Default ServiceAccount per namespace — no cluster-admin privileges
- Pods use default SA unless explicitly assigned a role

- IRSA (IAM Roles for Service Accounts) — pods can assume specific IAM roles for AWS API access

7. Summary

Layer	Tool	Purpose
Code	gitleaks, Bandit, Semgrep	Catch bugs and secrets before merge
Dependencies	npm audit, pip-audit	Block known CVEs
Images	Trivy	Block vulnerable container layers
Secrets	External Secrets Operator	Never store secrets in Git
Deployment	ArgoCD	Auto-sync, self-heal, no manual kubectl
Registry	ECR lifecycle policies	Keep only verified images
Network	ClusterIP, future Ingress + NetworkPolicies	Restrict traffic flow
Identity	OIDC, IAM roles, RBAC	Least-privilege access